



Efficient retraining of machine learning algorithms in cloud management systems

Lidia Kidane ^{a,*}, Paul Townend ^a, Thijs Metsch ^b, Erik Elmroth ^a

^a Autonomous Distributed Systems Lab, Department of Computing Science, Umeå University, SE-90187, Umeå, Sweden

^b Intel Corporation, Neubiberg, Germany

ARTICLE INFO

Keywords:

Cloud computing
Resource provisioning
Workload prediction
Concept drift
Data mining

ABSTRACT

Cloud management systems performing capacity autoscaling, application orchestration, server consolidation, and service differentiation increasingly rely on machine learning (ML) models for predictive decision-making. However, changes in user behavior, software updates, and hardware upgrades cause monitoring data to deviate from the training distribution, leading to model performance degradation. This phenomenon-known as *concept drift* poses a significant challenge to maintaining prediction accuracy in dynamic cloud environments. In this work, we propose a hybrid concept drift detection approach that combines statistical data monitoring with model performance metrics to inform efficient model retraining. The proposed method minimizes false positives, reduces adaptation delays, and avoids unnecessary retraining during stable periods. We conduct extensive experiments on both synthetic and real-world cloud workload datasets collected from multiple data centers, evaluating the hybrid approach against five established drift detection algorithms. To ensure statistical rigor, all experiments are repeated ten times, and the results are reported with 95% confidence intervals and significance tests. The results show that our proposed approach and implemented methods improve drift detection efficiency by eliminating unnecessary retraining and lead to more than 60% improvements over the baseline prediction accuracy.

1. Introduction

Cloud computing is a widely used model where services and resources are provided to users on request and charged using a variety of payment models [1]. An important characteristic of Clouds is elasticity - the ability of a system to adapt to workload changes by automatic provisioning and de-provisioning of resources [2]. Cloud management systems now depend heavily on machine learning algorithms to optimize dynamic tasks such as autoscaling, application orchestration, server consolidation, and service differentiation. Shifting from reactive to proactive management, machine learning enables cloud environments to adapt seamlessly to varying demands, making it essential for intelligent and automated cloud operations. *Workload forecasting* allows service providers to plan ahead to eliminate the problem of *under-provisioning*, where not enough resources are allocated to users, resulting in a breach of Service Level Agreement (SLA) and degradation of Quality of Service (QoS). Conversely, allocating excess resources to a user can result in *over-provisioning*, which leads to resource and energy wastage. Future workload prediction is therefore a key management tool to assist in speedy elasticity and on-time resource provisioning within Clouds.

This is particularly useful when managing virtualized resources such as Virtual Machines (VMs) [3] and container-based microservices [4].

Cloud environments generate continuous streams of monitoring data-such as CPU usage, network traffic, and application performance metrics. Those metrics are utilized by ML models to make predictive, data-driven decisions. By analyzing this real-time data, machine learning models can anticipate workload fluctuations, optimize resource allocation, and prevent performance bottlenecks, ultimately reducing costs. Cloud environments often incorporate stream processing for monitoring applications and resources, enabling real-time analytics, as shown in [5]. This technology plays a crucial role in orchestrating cloud systems by leveraging the most up-to-date information about workloads. Service providers harness multiple data streams from observability stacks to build accurate predictive models based on machine learning [6], aiming to understand and predict system behavior. However, cloud infrastructures and processes are inherently dynamic, as noted in [7] and [8]. Factors such as evolving user needs and changing application requirements contribute to gradual variations in workload patterns over time, resulting in changing data distributions. In this context, “concept drift” (CD) [9] which is defined as unforeseen changes in statistical properties

* Corresponding author.

E-mail addresses: lkidane@cs.umu.se (L. Kidane), paul.townend@umu.se (P. Townend), thijs.metsch@intel.com (T. Metsch), elmroth@cs.umu.se (E. Elmroth).

<https://doi.org/10.1016/j.future.2026.108478>

Received 31 March 2025; Received in revised form 24 January 2026; Accepted 7 March 2026

Available online 11 March 2026

0167-739X/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

of data over time is a significant concern; it leads to the obsolescence of machine learning (ML) models as out of distribution (OOD) inputs emerge [10]. Such changes manifest as statistical shifts in data, causing a disparity between the distribution of a training set and its related test set, ultimately resulting in a decline in model prediction performance [11]. Thus, *workload prediction models utilized in cloud management need to adapt to OOD data emerging in streams*. To ensure timely adjustments and seamless adaptation to changing conditions, models need to be monitored and adapted to changes regularly [12], ideally with minimal user intervention. This is crucial for effectively managing and scaling cloud resources per current requirements and criteria.

Various methodologies are used to adapt deep learning models to emerging concepts [13], with model retraining using newly arriving data widely adopted. Works including [14] and [15] proposed concept drift-based adaptation for models predicting cloud workloads. However, those works rely solely on the decisions of concept drift detection algorithms, which are prone to false positives due to noise and random patterns. Additionally, the algorithms exhibit a delay in detecting concept drift in workloads [16], thereby delaying the production of a new working prediction model. The primary objective of this study is to devise approaches for retraining models in such contexts that reduce computational overhead, minimize storage requirements, and enhance efficiency. To solve this problem, we introduce a hybrid adaptive workload prediction approach designed for dynamic cloud environments. Our approach comprises two steps to detect concept drift rather than relying on a single method. First, the concept drift detection algorithm rigorously examines statistical shifts in incoming data. Next, a performance-driven strategy is employed to monitor model prediction errors. These two robust methods serve as a dual-validation system to detect concept drift, ensuring accuracy rather than relying on a single approach. Additionally, the method leverages warning signals to gather data for subsequent model adaptation, enhancing its responsiveness and effectiveness. This approach to detecting concept drift makes our method highly resilient to false positives and significantly reduces delays in the adaptation process. Our research involves conducting experimental assessments to explore efficient strategies for updating machine learning models used in cloud management systems, where workload patterns change within the system [17]. Moreover, we explore how variables such as data granularity, data type, and window length influence adjustments to model parameters and their subsequent implications for resource constraints.

1.1. Contributions

This work is an extension of our previous work [18], where we implemented and explored ways to retrain machine learning models utilized on cloud management systems. This work improves the previous work by adapting automated monitoring for concept detection and an adaptation framework that continuously assesses model performance and triggers retraining when concept drift is detected. Moreover, we propose a new concept drift detection method Hybrid concept drift detection, for efficient retraining of machine learning models utilized in cloud management systems. In contrast to the widely used adaptation technique to concept drift in production environments, our method ensures timely updates without unnecessary retraining in stable periods. On top of the automated system, asynchronous retraining is utilized, where the model is updated in the background while the existing version continues to make predictions. This reduces downtime and allows for a seamless transition to the updated model. We delve into the concept of retraining ML-based cloud management systems using a drift-based approach, conducting comprehensive evaluations that consider characteristics commonly observed in cloud workload datasets. By performing this evaluation, we contribute to advancing the understanding of concept drift in cloud management systems and offer practical insights to improve predictive efficiency amidst shifting statistical distributions. Our discoveries not only illuminate the efficacy of current techniques but also tackle concerns about their adaptability to cope with the unique data char-

acteristics found in cloud environments. Ultimately, our objective is to reinforce the performance and dependability of cloud management systems by providing valuable guidance on the timing and methodology for retraining machine learning models, as well as the sensible selection of data for this task. An overview of the system workflow is illustrated in Fig. 1 In our evaluations, we establish a static workload predictor as the baseline and conduct comparisons with adaptive methods. We expand upon the predictor by incorporating five alternative techniques for identifying situations where retraining becomes necessary. In summary, this paper's primary contributions are as follows:

1. We propose an informed hybrid concept drift detection approach to adapt workload prediction models when faced with concept drift in cloud workloads. Considering that concept drift detection algorithms often yield false positives and are susceptible to noise in data, the proposed approach integrates data monitoring with model performance information. This integration aims to trigger model updates, ensuring efficient adaptability in response to changing conditions.
2. We demonstrate efficient retraining approaches based on a set of dimensions in evaluation, taking into account factors such as the delay in detecting workload changes, prediction errors in the updated model, and the overall efficiency of the adaptation process. These evaluations encompass data granularity, data selection for model update, magnitude of concept drift in data, and corresponding computational overhead to detect and adapt to concept drift in the data.
3. Evaluations are based on synthetic and real-world scenarios using actual datasets acquired from two data centers. The outcomes of our experiments demonstrate that informed concept drift adaptation has the potential to yield up to a 60% improvement compared to the non-adaptive baseline prediction.

The rest of this paper is organized as follows. Recent related work on cloud resource prediction and the issue of adaptability is summarized in Section 2. In Sections 3 and 4, our methodology and experimental setup are discussed in detail. Finally, result evaluations and conclusions are discussed in Sections 5 and 6, respectively.

2. Related work

Research on workload prediction in cloud computing, categorized as classical machine learning, and deep learning approaches. Furthermore, we will highlight advancements in the field of concept drift detection and adaptation, both in a general context and specifically within the domain of cloud computing

2.1. Classical machine learning-based methods

In [19] the authors proposed workload prediction based on an ensemble approach that combines multiple models, where prediction is done based on the result of various predictors. The ensemble layer holds a residual layer to give feedback for proper optimization. Liu et al. [20] proposed an adaptive workload prediction that first categorizes workloads into different classes and assigns them to other prediction models according to workload features. A clustering-based learning is proposed by [21] where a pool of jobs is clustered by workload characteristics, and a neural network-based predictor is used on the clusters.

2.2. Deep learning-based methods

With recent advancements in deep learning, researchers have used these models to predict workload and achieve efficient, accurate results. An autoencoder-based prediction is proposed in [22] to extract features from the complex nature of workloads and achieve accurate prediction. Kumar et al. presented a workload prediction approach based on Long Short-Term Memory (LSTM) and Recurrent Neural Network (RNN), and Song et al. [23] proposed an LSTM model to predict mean load over various time intervals. and Bidirectional Long-Short Term Memory (BLSTM)

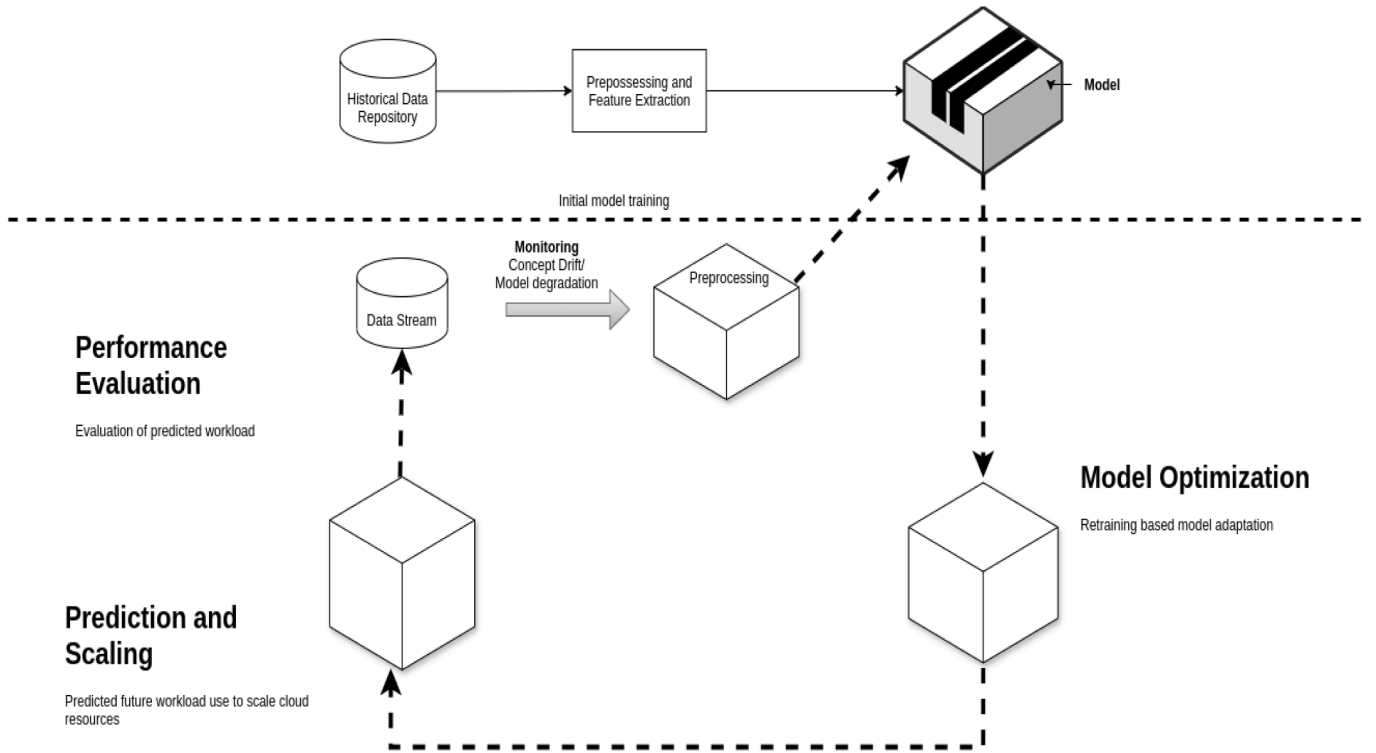


Fig. 1. Proposed system architecture for Efficient retraining of machine learning in cloud management.

based on association learning to capture the relationship between resource metrics is proposed by [24] for workload prediction and task failure prediction in cloud systems by [25].

Mason et al. [26] proposed an evolutionary neural network to predict host utilization accurately in ever-changing resource consumption scenarios. Duggan et al. [27] combine linear and nonlinear prediction techniques with RNN to network bandwidth and CPU utilization. Moreover, to achieve adaptive workload prediction in highly variable workloads, Chen et al. [28] combines a top-sparse auto-encoder for dimension reduction and Gated Recurrent Unit (GRU) for prediction.

Saxena et al. [29] proposed a method to adaptively learn workload traces by using an auto-adaptive differential algorithm to forecast average workloads. Conversely, Bao et al. [30] proposed a bagging-like ensemble framework for cloud workload prediction with adaptive pattern mining. They develop a two-step method to capture characteristics of highly variable workloads and error-based weight aggregation to integrate prediction results from various models.

2.3. Adaptation strategies for concept drift

Adaptation strategies for concept drift play a crucial role in ensuring the continued accuracy and reliability of predictive models in dynamic environments. As the statistical properties of data change over time, concept drift introduces challenges that necessitate the development and implementation of effective adaptation strategies. These strategies aim to enable models to adapt and adjust to evolving patterns, thereby maintaining their predictive power in the face of concept drifts.

Adaptation strategies for concept drifts encompass a wide range of techniques and approaches [31]. One common strategy involves online learning, where models are continuously updated with incoming data to capture and reflect the changing underlying concepts. This approach allows the model to adapt incrementally and maintain up-to-date knowledge, reducing the impact of concept drifts on prediction accuracy.

Another strategy involves the use of ensemble learning [32], where multiple models are combined to improve robustness and resilience

against concept drifts. Ensemble techniques leverage the diversity of individual models to capture different aspects of the evolving data distribution, enhancing the overall predictive performance. Furthermore, transfer learning approaches are utilized as adaptation strategies [33,34], leveraging knowledge from previously seen domains to aid in the adaptation to new and unseen data distributions. By transferring relevant knowledge and leveraging pre-existing models, transfer learning can accelerate the adaptation process and enhance the model's ability to handle concept drifts. Moreover, Baier et al. [35] presented data selection mechanisms to retrain machine learning models in the presence of concept drift in business process mining.

Ultimately, successful adaptation strategies for concept drifts require a combination of continuous monitoring, timely detection, and effective adjustment techniques. The choice and effectiveness of these strategies depend on the specific application domain, the characteristics of the data, and the anticipated nature of concept drifts. By incorporating robust, adaptive mechanisms into predictive models, practitioners can mitigate the detrimental effects of concept drift and ensure accurate predictions in dynamic, evolving environments.

The previously mentioned studies focus on predicting future workloads' accuracy, but do not account for the dynamic nature of workloads. There have been works in adaptive workload prediction approaches, including [36], where a prediction model based on episode mining and online learning is proposed, leading to adaptability capability to the prediction mode. Toumi et al. [37] utilized stream mining techniques along with an ensemble learning prediction approach to adapt to changes in server load, and [15] proposed a CNN and LSTM-based model to predict faults in dynamic edge clouds that incorporate concept drift detection and adaptation strategies. Mallick et al. [38] proposed a similarity metric to find the most similar data batch to use the corresponding ML model for prediction in the presence of data drift in large-scale systems. Moreover, Shayesteh et al. [39] present an automated reinforcement-learning approach for handling concept drift in fault prediction in edge clouds. Nonetheless, none of the previously mentioned approaches have addressed the critical aspects of data granularity and the selection of

Table 1
Hyperparameters and selected values
utilized for grid search.

Hyper-parameter	values
Batch size	32, 64, 128
Epochs	50, 100, 150
Number of neurons	32, 64, 128
Learning rate	0.001, 0.01, 0.1

data to use for model retraining. In our work, beyond the initial adaptability criteria, we explicitly incorporate these factors to propose an efficient model update strategy for responding to workload changes in dynamic environments. The overall system workflow is shown in Fig. 1.

3. Methodology

In this section, we present the overall flow of the algorithms we implemented to test our hypothesis. Firstly, we implement a workload prediction model trained from historical workload traces to predict future workload. As context changes in future workload patterns, the old model needs to be updated to reflect the new concept and the latest state of the system.

As shown in Fig. 1, the architecture comprises initial LSTM model training based on historical workload and in-production model usage. The components in the in-production phase include a future workload prediction generator, concept drift detector, performance monitor, and retraining modules. The concept drift detector analyzes new workloads for pattern changes, and, upon detecting drift or performance degradation, triggers model optimization.

3.1. Prediction model

The LSTM [40] model serves as the baseline prediction model in all experiments. Recurrent Neural Networks (RNNs) is a special type of Artificial Neural Network (ANN) with the ability to pass signals in both directions, designed to learn sequential patterns [41]. The downfall of RNNs is that they exhibit a vanishing gradient problem when the distance from the input is long. However, LSTM is proposed to overcome the vanishing gradient problem by integrating memory blocks rather than summation units, as in RNNs. Moreover, it shows the ability to capture long-term temporal dependencies in data [42]. For instance, cloud workload trace datasets show long-term seasonality and patterns. While alternative models such as Gated Recurrent Units (GRUs), ARIMA, and Transformer-based predictors have shown promise for time series forecasting, previous studies [43,44] indicate that LSTMs offer a good balance between complexity and performance for workload traces. For consistency and comparability with prior research, we therefore employ LSTM as the baseline predictor. Future work will explore the inclusion of GRU and Transformer-based architectures for further benchmarking.

Reviewer 2, comment 1: We have now expanded the justification for selecting LSTM as the baseline predictor in Section 3.1 (Prediction Model). Specifically, we added a discussion comparing LSTM with alternative approaches such as GRU, ARIMA, and Transformer-based models, highlighting trade-offs in complexity, data requirements, and suitability for cloud workload traces. We clarify that LSTM was selected due to its strong performance in prior cloud workload studies and its balance between modeling long-term dependencies and computational efficiency. We also explicitly note that benchmarking GRU and Transformer-based predictors is an important direction for future work, which is now stated in both Section 3.1 and the Conclusion.

We utilize the LSTM model as the baseline method. To find the optimal hyperparameters, we perform a grid search and select the parameters that lead to the minimum RMSE (Root Mean Squared Error) values (see Section 4.2) when applied to various dataset types, as shown in Table 1. Thus, optimal parameters are utilized over all the experiments.

3.2. Concept drift detection methods

The workload change point detection part comprises algorithm that monitor streaming workload data for possible concept drifts that arise due to various factors. We implemented five concept drift detection algorithms that illustrate state-of-the-art performance in detecting drifts in regression tasks [36]. Moreover, we proposed a *hybrid approach* to detect change point in workload that blend the functionality of above-mentioned methods to efficiency of the working model. This approach helps eliminate the possibility of detecting false positives, as the above-mentioned methods are also prone to outliers in the data. To evaluate the performance of the drift detection methods, metrics including drift detection delay, number of false positives detections are utilized.

3.2.1. Adaptive windowing (ADWIN)

[45] A slide window-based approach to detect concept drift in streaming data. Instances of normalized data in the range $x \in [0, 1]$ are inserted into the variable-size window as long as the distribution in the data remains constant. Data is further divided into sub windows W_0, W_1 , then ADWIN computes the average of two windows and compares the difference to assess the change in distribution between the sub-windows. If the difference in mean between two sub-windows exceeds a user-defined threshold $\delta \in (0, 1)$, a concept drift alarm is generated. Full pseudocode is illustrated in Algorithm 1:

Algorithm 1: ADWIN: Adaptive sliding window algorithm.

Data: Input data stream $x_1, x_2, \dots, x_n$, Confidence threshold ϵ_{cut}
Result: Estimate of mean $\hat{\mu}_W$

- 1 Initialize sliding window W ;
- 2 **for** $t = 1$ **to** n **do**
- 3 Add example x_t to sliding window W ;
- 4 Calculate the average $\hat{\mu}_W$ of examples in W ;
- 5 **repeat**
- 6 Drop elements from the tail of W ;
- 7 Update $\hat{\mu}_W$ accordingly;
- 8 **until** $|\hat{\mu}_W - \mu_t| \geq \epsilon_{cut}$;
- 9 **if** $|W| > 1$ **then**
- 10 Split W into two sub-windows W_0 and W_1 such that
 $n = n_0 + n_1$;
- 11 **Output** $\hat{\mu}_W$;

3.2.2. Page hinckley (PH)

[10] An extension of the Cumulative Sum (CUMSUM) algorithm, which computes a running mean across data points. Whenever a new element arrives in a stream, the running mean is updated. If the difference between the minimum and current mean exceeds a user-defined threshold (λ), a concept drift is detection alarm generated as shown in Algorithm 2:

3.2.3. Drift detection method (DDM)

[10] this algorithm tracks model prediction errors in streaming data. This drift detector keeps track of two variables p_i and σ_i corresponding to current model error and standard deviation respectively. On each prediction of a new example, $p_{\min}$ and $\sigma_{\min}$ are calculated corresponding to minimum error and standard deviation. Confidence for drift detection is set to $p_i + \sigma_i \geq p_{\min} + 3 * \sigma_{\min}$ and warning level to $p_i + \sigma_i \geq p_{\min} + 2 * \sigma_{\min}$ respectively.

3.2.4. Early drift detection method (EDDM)

[46] This method detects concept drift by computing the average distance of 2 consecutive errors to generate a warning and alarm. It tracks the running mean distance p_i and standard deviation σ_i of the errors and compares the maximum mean $p_{\max}$ and standard deviation $\sigma_{\min}$ to

Algorithm 2: Page hinkley test.

Data: $x_0, x_1, x_2, \dots, x_T$

- 1 Acceptable Distance = σ ;
- 2 Detection threshold = λ ;
- 3 **while** $t > 0$ **do**
- 4 $\mu_T \leftarrow 1/T \sum_{i=0}^T x_i$ ▷ * [r] Calculate a running mean
- 5 $U_T \leftarrow 1/T \sum_{i=0}^T (x_i - \mu_T - \sigma)$;
- 6 $m_T \leftarrow \min U_T$
- 7 $PH_T \leftarrow U_T - m_T$
- 8 **if** $PH_T > \lambda$ **then**
- 9 Change detected at t ;
- 10 **else**
- 11 Return to step 1 ;

a user-defined threshold to generate alarm and concept drift detection. Drift alarm is generated on $\frac{p_i + 2\sigma_i}{p_{\max} + 2\sigma_{\max}} \leq \alpha$ while drift detection is set on $\frac{p_i + 2\sigma_i}{p_{\max} + 2\sigma_{\max}} \leq \beta$ where, α and β are user defined thresholds.

3.2.5. Prediction error (PE)

This method works by tracking the prediction error of the original predictive model and comparing it with a user-defined threshold. The parameter depends on the user's preference. In situations where only the direction of the data change is needed to be known, such as in cloud management systems, a higher threshold is set. In contrast, in environments where accurate predictions are in need, a smaller threshold is set which is sensitive to small changes in the prediction of the model. To detect concept drift occurrences, we utilize the equation provided below.

$$\left| \frac{1}{n} \sum_{i=1}^n (\text{predicted}_i - \text{real}_i) \right| > \text{threshold}$$

Where predicted_i represents the i th predicted value in the list, n is the total number of samples selected for test, and threshold is the pre-defined acceptable prediction error limit produced by the model. Full pseudocode for PR based change detection and adaptation is illustrated in [Algorithm 3](#):

3.2.6. Hybrid concept drift detection

Here, we propose a hybrid approach to detect and adapt to concept drift. This method integrates data monitoring and model prediction performance evaluation to detect and adapt to changes in the data distribution over time. The overall workflow is illustrated in [Algorithm 4](#). This work is an improvement of [47] that combines decisions from a method that detects drifts directly from the time series data along with the model forecasting error. Hence, we integrate findings from both ADWIN concept drift detection and LSTM model prediction error. Detecting concept drift in the data produces a warning, which subsequently initiates the data collection process for potential model updates later. Confirmation of the drift warning occurs upon the arrival of real data y for model predictions $\hat{y}$ in the stream, and an error is computed. Once the prediction error exceeds a predefined threshold, model retraining is triggered with the new data batch in the window W . The improvement in this approach over the aforementioned individual data monitoring or model prediction error monitoring-based detection approaches is twofold. Firstly, data monitoring approaches solely focus on incoming data, rendering them susceptible to false positive detections arising from noise and outliers. Furthermore, model error-based approaches must await the arrival of real data in the stream to detect drift, resulting in delays in adaptation. However, by combining both approaches, these issues are inherently addressed. This combination eliminates unnecessary retraining due to false positives by confirming them and initiates the data collection process

Algorithm 3: Update LSTM model with sliding window.

Input: Pre-trained LSTM model $\mathcal{M}$, new data batch $D = \{x_t\}_{t=1}^T$, error threshold ϵ

Output: Updated LSTM model $\mathcal{M}'$

- 1 Load the pre-trained LSTM model $\mathcal{M}$;
- 2 Initialize a sliding window of size w over the time series data D ;
- 3 **while** *True* **do**
- 4 Extract the current window of data
- 5 $D_w = \{x_t, x_{t+1}, \dots, x_{t+w-1}\}$;
- 6 Make predictions $\hat{y}_t = \mathcal{M}(D_w)$ using the pre-trained model on D_w ;
- 7 Compute the prediction error:
- 8
$$E_t = \frac{1}{w} \sum_{i=t}^{t+w-1} (y_i - \hat{y}_i)^2$$
- 9 where y_i is the actual data in the window and $\hat{y}_i$ is the model's prediction;
- 10 **if** $E_t > \epsilon$ **then**
- 11 Update model parameters: $\mathcal{M}' = \text{Update}(\mathcal{M}, D_w)$ using backpropagation through time (BPTT);
- 12 Save a checkpoint of the updated model $\mathcal{M}'$;
- 13 Slide the window forward by removing the oldest data point x_t and adding the next data point x_{t+w} ;

early, facilitating timely adaptation. Full pseudocode for hybrid concept drift detection and adaptation is illustrated in [Algorithm 4](#):

Algorithm 4: Hybrid concept drift detection and adaptation.

Data: Initial trained model, new data, streaming data, predicted data

Result: Updated model

- 1 **while** *new incoming data* **do**
- 2 Feed data to concept drift detection method;
- 3 **if** *detector generates warning* **then**
- 4 Start saving data to window W ;
- 5 **else if** *concept drift detected* **then**
- 6 Initial model predicts next value;
- 7 Compute error (predicted, true value);
- 8 **if** *concept drift detected (error greater than threshold)* **then**
- 9 Retrain with window W and new data;
- 10 Save model;
- 11 Empty window W ;
- 12 **else**
- 13 Return to step 1;

3.3. Concept drift adaptation

Management of ML models is a crucial task due to changes in the data distribution over time in real-world scenarios [48]. Batch learning is widely used with the assumption that data distribution persists stationary over time [49]. In order to make ML models in production robust to concept drift, two approaches can be followed, informed and blind adaptations [11]. The informed adaptation incorporates information from data mining algorithms to back the decision of model update. On the other hand, blind adaptation includes online learning approaches where the model is updated on the fly with the arrival of every new instance in the stream. The informed approach has the capability to save com-

putational costs by eliminating unnecessary retraining decisions while the data distribution remains stationary. In order to reduce the potential unnecessary computational load and gain an understanding of the nature of the drift, we follow an informed concept drift adaptation method in this work. As illustrated in Algorithm 5, this work follow the batch learning approach to update the old model with the new concepts [10]. The model parameters are updated based on the equation below.

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{1}{n} \sum_{i=1}^n \nabla J(\theta_{\text{old}}, \mathbf{x}_i, y_i)$$

In this scenario, θ_{old} represents the parameters of the model before retraining, θ_{new} represents the updated parameters of the model after retraining, α is the learning rate, n is the total number of data points in the retraining batch, $\mathbf{x}_i$ is the feature vector of the i th data point in the retraining batch, y_i is the corresponding label of the i th data point in the retraining batch, and $J(\theta_{\text{old}}, \mathbf{x}_i, y_i)$ is loss, and finally, ∇ denotes the gradient operator.

Algorithm 5: Model retraining for concept drift adaptation.

Input: Initial model with previously trained wights θ , Data stream
Output: Updated model

```

1 while True do
2   Collect a batch of the most recent data  $D_t$ , where  $t$ 
   represents the time step;
3   Detect concept drift  $CD(D_t)$ ;
4   if Concept drift detected then
5     Preprocess the new data batch;
6     Update the model parameters  $\theta$  using the new data
       batch:  $\theta_{t+1} = \text{Update}(\theta_t, D_t)$ ;
7     Evaluate the updated model;
8   Monitor for concept drift;
```

4. Experimental setup

To investigate the performance of an adaptive workload predictor using five different methods for drift detection, we use an LSTM model as the baseline predictor. The adaptive predictor is created by expanding upon the baseline predictor, triggering retraining whenever prompted by a drift detector. For evaluation purposes, this retraining process is carried out using five distinct drift detectors.

Experiments are performed on a machine with specifications, Intel Core i7-7700 3.6 GHz CPU with four cores and 32GB of RAM. Additionally, LSTM model is implemented with Keras¹ and Tensorflow² backend whereas stream processing algorithms are based on scikit-multiflow³ and River⁴ stream machine learning packages.

4.1. Datasets

To evaluate the methods, we utilize synthetic and real-world cloud workload datasets across all the experiments. The synthetic datasets have predefined concept drifts and various characteristics that are induced intentionally to thoroughly evaluate the methods in all aspects. Moreover, real world workload datasets are utilized to evaluate the effectiveness of the methods in real world scenarios. CPU utilization and memory utilization are workload metrics utilized for forecasting in this scenario.

¹ <https://keras.io/>

² <https://tensorflow.org/>

³ <https://scikit-multiflow.github.io/>

⁴ <https://riverml.xyz/>

Table 2

Real-world cloud workload dataset-1 properties.

Field	Min	Max	Mean
CPUMinLoad Mhz	0.0	33267.2	18941.9
CPUMaxLoad Mhz	4990.0	69029.4	26466.5
CPUAvgLoad Mhz	831.7	36593.9	21775.3
MemMinLoad Mhz	0.0	137.2	115.5
MemMaxLoad Mhz	8.4	137.23	115.6
MemAvgLoad Mhz	8.3	137.2	115.6

Table 3

Real-world cloud workload dataset-2 properties.

Field	Unit	Min	Max	Mean
CPU utilization	%	7.0	80	17.9
Memory utilization	%	3.3	4.8	4.01

4.1.1. Synthetic workload

We generated synthetic time-series datasets using sinusoidal signals [16] consisting of trend, seasonality, and noise to match the properties of cloud workload datasets. The datasets are injected with the four concept drift types namely, sudden, incremental, gradual, and recurrent. Moreover, the magnitude of change in the data varies by 25%, 50%, and 100% to test the performance of implemented concept drift detection algorithms on various levels of change in the data. These changes alter the statistical properties of the dataset.

Furthermore, to simulate varying data granularity levels in collecting cloud workload traces and to assess the effect of the frequency at which observations are recorded, we introduced various season lengths in the data. We manipulate the dataset by adjusting the number of instances within a season. This results in seasons containing 50 instances, 100 instances, and 150 instances, respectively. These synthetic datasets enable us to evaluate the performance of drift detection methods across a spectrum of behaviors commonly exhibited in cloud workload datasets.

4.1.2. Real-world workload 1

A real-world dataset collected from a UK based data-center (details available on request) is utilized. Traces from 59 servers are collected. Initially, we collected traces from a total of 59 servers. Afterward, we filtered out traces that had fewer than 2000 collected instances and those that did not display any significant changes in workload characteristics over time. To identify the traces demonstrating concept drift, we applied the ADWIN and pH algorithms. This selection process resulted in us choosing 9 servers for conducting comprehensive experiments. The dataset consists of workload metrics collected at different time intervals, such as daily, hourly, and every 5 minutes. To create uniform daily workload traces for analysis, we aggregated the instances. Within this dataset, we identified instances of both abrupt and gradual concept drift occurring at various levels and positions. To rigorously evaluate the methods and test their robustness, we used the dataset in its entirety and also selectively removed specific instances to manipulate the data collection frequency. As illustrated in Table 2, the traces collected have 6 fields. CPUAvgLoad is utilized in the experiments. Thus, a model is trained with this metric to predict future trends. Traces from one server is shown in Fig. 2.

4.1.3. Real-world workload 2

A real-world dataset collected from Ericsson Sweden data center. As illustrated in Table 3, we select two metrics from the traces; CPU utilization and memory utilization for prediction. The duration of the dataset is two months. One instance of traces for memory utilization metric is shown in Fig. 3.

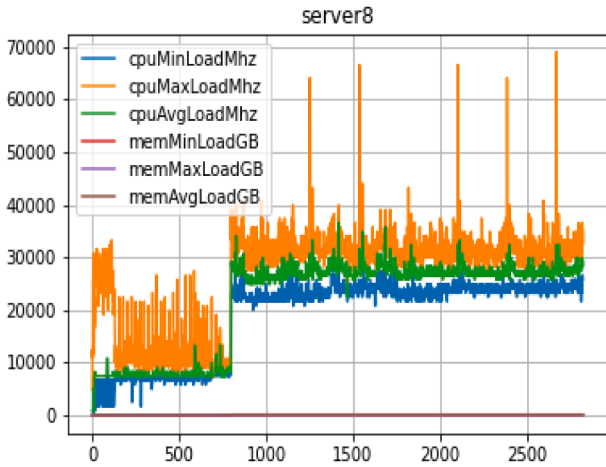


Fig. 2. Workload characteristics in real-world traces 1.

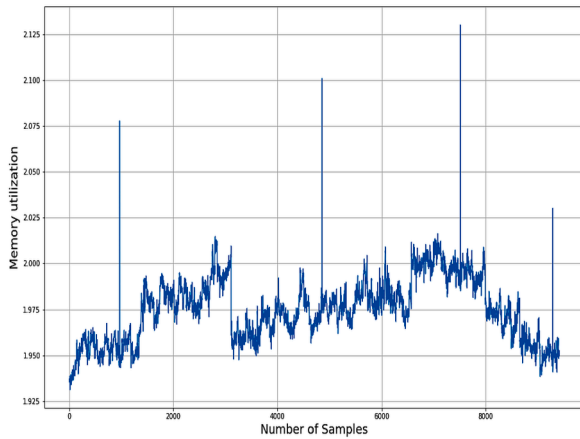


Fig. 3. Workload characteristics in real-world traces 2.

4.2. Evaluation metrics

To assess prediction performance in our evaluation, we employ several key metrics, namely Root Mean Squared Error (RMSE), Normalized Root Mean Squared Error (NRMSE), and Mean Absolute Percentage Error (MAPE). These metrics serve as consistent benchmarks throughout our experiments, enabling us to gauge the performance of our methods effectively.

1. Root Mean Square Error (RMSE)

$$RMSE = \sqrt{\left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - x_i)^2}$$

2. Normalized Root Mean Square Error (NRMSE)

$$NRMSE = \left(\frac{RMSE}{(y_{\max} - y_{\min})}\right)$$

3. Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{1}{n} \sum_{i=1}^n 100 \left| \frac{x_i - y_i}{x_i} \right|$$

where x_i and y_i denote true values and predicted results respectively and n represents the number of samples.

4.3. Hyperparameter setting

ADWIN and pH examine the incoming data and generate detection and warning on concept drift detection. On the other hand, EDDM and DDM monitor model predictions. All the concept drift detection algorithms are set with default thresholds based on Scikit-multiflow framework [50]. Where $\delta = 0.002$ is set for ADWIN and, on the other hand, pH is set with $\delta = 0.001$ and threshold $\lambda = 1$. Performance-based concept

drift detectors DDM, EDDM, and PE are utilized. DDM is set with parameters minimum instances $n = 50$, warning level $w = 2.0$, and out control level $c = 3.0$ and EDDM is set with $\alpha = 0.95$ and $\beta = 0.9$. Whereas, the PE method is set with a threshold value of $NRMSE = 0.1$. As NRMSE metric normalizes the RMSE error to be between 0 and 1, 0.1 denotes approximately 10% error in the predicted data. Thus, we set the threshold to initiate retraining to 10% error in the predictions.

To assess robustness, we performed a sensitivity analysis by varying drift detection thresholds by $\pm 20\%$ around their default values. The hybrid method remained stable for $\delta \in [0.0015, 0.0025]$ and $\lambda \in [0.8, 1.2]$. Within this range, detection delay and false positive rates varied by less than 5%. Outside these bounds, purely statistical detectors such as ADWIN became more sensitive to noise, whereas the hybrid method maintained stable performance due to its dual validation mechanism.

Review 1, comment 4: This is an important point. We have now added a hyperparameter sensitivity analysis in Section 4.3, in which detection thresholds are varied by $\pm 20\%$ around their default values. The results show that the hybrid method remains stable, with detection delay and false positive rates varying by less than 5%. We clarify that manual tuning is not required: default parameters generalize well across datasets, and the hybrid design further reduces sensitivity compared to purely statistical detectors. We understand that this is a significant issue in new deployments, and we have dedicated a new study to hyperparameter tuning in this work [51]. Thus, this study falls outside the scope of this work.

5. Evaluation

In this section, we present the results and corresponding evaluations of our proposed method. Initially, we assess various concept drift detection methods using both synthetic and real-world cloud workload datasets. Following that, we delve into the workload prediction performance, comparing the baseline approach with adaptive models re-trained based on informed decisions from drift detection methods. To provide clarity and structure to our evaluations, we categorize them based on the datasets used in our experiments: synthetic datasets, and real-world datasets, organized under the themes of **concept drift detection** and **adaptation**.

The key findings in the evaluation include:

- All employed methods successfully detected all change in patterns in workload and in turn initiated model update when it was necessary.
- Model performance-based detectors showed a delay in detection. This affects both the efficiency and the model accuracy in the long run.
- Data appeared in the stream before concept drift detection notification showed to be useful in most cases.
- Hybrid approach that combines statistical-based detectors with model error monitoring exhibited the most efficient and accurate approach of model update.

5.1. Synthetic dataset

5.1.1. Concept drift detection

In this section, we evaluate the effectiveness of concept drift detection methods by examining their performance, false positive detection rate, and detection delay across various magnitudes of data change. We introduce shifts of 25%, 50%, and 100% in magnitude relative to the original dataset settings within the synthetic data. Furthermore, we introduce abrupt drifts at instance 4000 within the datasets. Table 4 offers a comprehensive overview of the number of drift detections and the corresponding detection delays for each of the following algorithms: ADWIN, PH, DDM, EDDM, PE, and Hybrid. Delays are represented as both positive and negative, where positive delay indicates detections occurring after the drift in the data, and negative delay implies detections happening before the drift. As depicted in Table 4, the detection

Table 4

Drift detection by ADWIN, pH, DDM, EDDM, PE and Hybrid methods on percentage change magnitude 25%, 50% and 100%. Drift is induced at instance 4000 for all the datasets.

Change %	Algorithm	Total Detections	Delay
25%	ADWIN	1	+115
	pH	1	+172
	DDM	3	+1920, +2800, +3700
	EDDM	0	unavailable
	PE	6	-182, +20
	Hybrid	1	+115
50%	ADWIN	1	+179
	pH	1	+95
	DDM	3	+1230
	EDDM	1	+505
	PE	3	-20, +35, +2280
	Hybrid	1	+179
100%	ADWIN	1	+83
	pH	1	+81
	DDM	1	+680
	EDDM	1	+470
	PE	3	-1820, +10, +920
	Hybrid	1	+83

delays for each method decrease as we transition from smaller shift percentages to larger ones. This observation suggests that smaller magnitude changes are consistently more challenging for the algorithms to detect promptly. Nevertheless, when examining performance monitoring-based algorithms such as EDDM and DDM, a clear enhancement in their performance becomes evident as we move from a 25% shift to a 100% shift in the data. In contrast, the statistical and window-based methods, specifically pH and ADWIN, consistently displayed robust performance across all scenarios, with only a slight decrease in performance noted when addressing the smallest data change. It is worth highlighting that the threshold-based method PE consistently demonstrated the shortest detection delay throughout the entire experiment. The evaluations are presented clearly in Fig. 4, where vertical lines indicate the index in the dataset where concept drift detection methods detected a drift. However, as Table 4 reveals, ADWIN, EDDM, DDM, and PE exhibit false positives in concept drift detection when dealing with 25% and 50% shifts. As the shift percentage increases to 100%, only PE registers 2 false positives, while the others perform correctly with no false alarms. pH consistently performs well across all shift levels.

In light of these experiments on the selected dataset, it becomes apparent that ADWIN and pH are relatively dependable methods with minimal performance degradation across various data shifts and only a few false positives. In contrast, performance-based methods like DDM and EDDM exhibit significant delays due to their reliance on model prediction error rather than changes in the data distribution within the stream. The PE method, while showing minor delays of +10 and +20 instances, also experiences multiple false positives in detection. This is because PE detects changes based on a simple drop in the performance of the prediction model. The hybrid method outperforms all other approaches in both scenarios, demonstrating no false positives and exhibiting a shorter delay.

Data granularity. To assess the performance of concept drift detection under varying data collection frequencies, we manipulate the granularity of the data by setting different levels. Specifically, we organize the data into seasons containing either 50 or 100 instances. This evaluation encompasses not only the examination of drift detection methods' performance but also an investigation into the impact of data granularity on the concept drift adaptation technique. We aim to determine whether the granularity level of the data affects the methods' capacity for drift detection and at what granularity level this effect becomes most pronounced. These evaluations will provide valuable insights into the

Table 5

Drift detection by ADWIN, pH, DDM, EDDM, PE and Hybrid on different data collection frequency (Season length: 50 and 100 instances) and 25%, 50%, and 100% data shifts.

Season Length	Algorithm	Delay		
		25% shift	50% shift	100% shift
50	ADWIN	79	111	207
	pH	94	144	246
	DDM	390	1220	280
	EDDM	280	2640	445
	PE	10	10	10
	Hybrid	79	111	207
100	ADWIN	83	179	115
	pH	81	95	172
	DDM	680	1230	1920
	EDDM	470	505	N.A.
	PE	10	35	20
	Hybrid	83	179	115

optimal level of detail at which to employ the drift detection methods for optimal performance within this domain.

Table 5 presents the performance of drift detection algorithms with season lengths of 50 and 100 instances, corresponding to shifts of 100%, 50%, and 25% in a synthetic dataset. Drifts are introduced at instances 2500 and 4000 for season lengths of 50 and 100 instances, respectively. Additionally, the tables display the detection delays exhibited by each concept drift detection algorithm across all data granularity levels, where 50 and 100 instances represent a season.

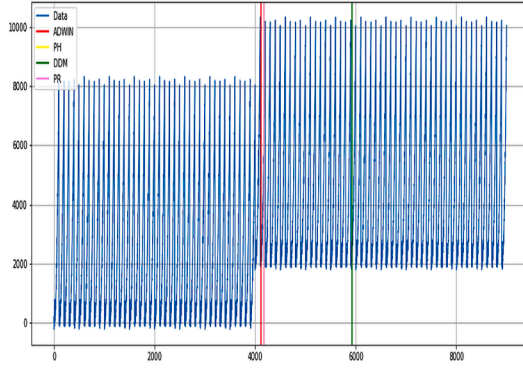
Notably, as the number of instances in a season increases, pH and ADWIN exhibit relatively stable performance without substantial changes. However, the performance of EDDM and DDM significantly deteriorates as the number of instances in a season grows to 100. In scenarios where a season contains fewer instances, the algorithms have a better opportunity to detect significant shifts in consecutive instances, leading to quicker detection. Conversely, in datasets with a higher number of instances per season, the changes occur more gradually, causing the algorithms to wait until these changes satisfy their predefined thresholds, resulting in detection delays.

Another noteworthy observation is that PE and Hybrid are not significantly affected by changes in season length across all cases, as indicated in Table 5. This phenomenon can be attributed to the methodology employed by the PE method. PE primarily focuses on the model prediction error of a window of instances, in contrast to DDM and EDDM, which consider running errors in prediction over a sequence of instances. This fundamental difference in approach allows PE to remain less influenced by changes in season length, as it assesses each window independently.

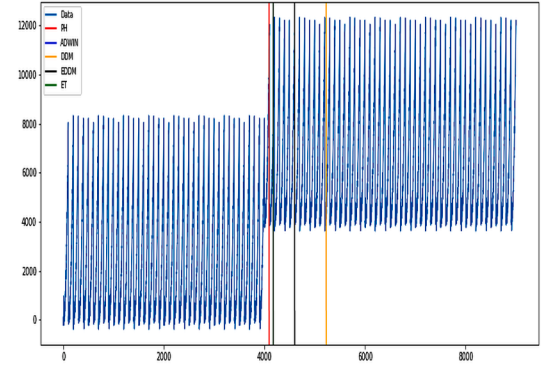
5.1.2. Concept drift adaptation

In this section, we delve into an examination of the algorithms, with a focus on their performance in adapting to concept drift. This analysis considers both the magnitude of change and the frequency of data collection. We begin by standardizing the data and rescaling it to fit within the range of 0 to 1. Subsequently, we partition the data into two subsets: a training set and a testing set, maintaining an 80:20 split ratio. The baseline LSTM model undergoes its initial training using the initial 1000 instances of the dataset. During this training phase, we carefully select hyperparameters through an exhaustive grid search process. The initiation of the model update is contingent upon notifications received from the concept drift detection methods. During the model update process, we employ various data window sizes, ranging from 100 to 1000 instances, to facilitate the adaptation of the model.

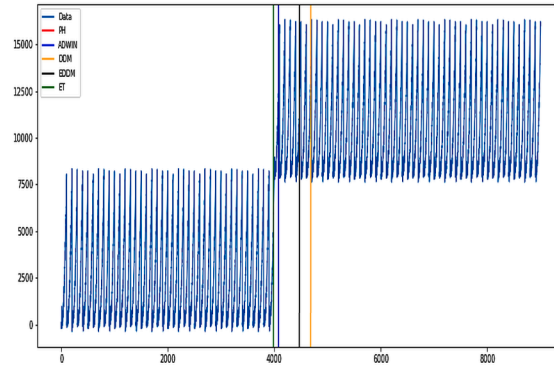
Sensitivity to the magnitude of drift. To assess the sensitivity of the workload shift's magnitude during the model retraining process, which includes evaluating the convergence time and the required number of instances, we supply the methods with data exhibiting varying degrees of change magnitude. The data collection frequency remains



(a) Concept drift detection at 25% shift and season length: 100 instances.



(b) Concept drift detection at 50% shift and season length: 100 instances.



(c) Concept drift detection at 100% shift and season length: 100 instances.

Fig. 4. Abrupt concept drift detection on synthetic workload data at various shift percentages with 100 season length detection by methods: ADWIN, PH, DDM, EDDM, and PE. The vertical lines indicate positions where drift is detected by each method.

constant, and we introduce shifts of 25%, 50%, and 100% from the original data setting. This evaluation aims to provide valuable insights into how the magnitude of the shift influences the performance of the model retraining process in terms of prediction accuracy and the delay in producing an updated model upon detecting concept drift. Table 6 provides insights into the performance of the prediction model in terms of MAPE% under two conditions: baseline static model and retraining based on concept drift detection using methods ADWIN, PH, DDM, EDDM, PE, and Hybrid. This analysis encompasses shifts of 25%, 50%, and 100% in the data magnitude. Notably, the results consistently demonstrate that the retrained model, guided by all the methods, surpasses the baseline model in performance where no retraining was applied. Furthermore, as we transition from a 25% to a 100% shift in the data, there is a substantial decrease in prediction error for the retrained model. This phenomenon can be attributed to the fact that the drift detection methods exhibit greater delays in detecting smaller shifts, as observed in Table 4. Consequently, the old model is utilized until model retraining is initiated. Moreover, it's important to highlight that the hybrid method not only outperforms all other approaches in detecting concept drift but also excels in the performance of the retrained model. In summary, larger shifts in data magnitude result in early detection by the methods, facilitating timely model retraining and leading to improved prediction performance.

Data granularity. To assess the impact of the chosen metric saving frequency, particularly with respect to the data granularity in the collected data, on model retraining performance and adaptation to change, we employ datasets with varying season lengths, specifically 50 and

Table 6

Drift adaptation on static model and retraining initiated by ADWIN, PH, DDM, EDDM, PE, and Hybrid on percentage shift magnitude 20%, 50% and 100%.

Data shift %	Algorithm	MAPE%
25	No retraining	227.49
	ADWIN	7.8
	pH	7.4
	DDM	7.2
	EDDM	unavailable
	PE	9.5
	Hybrid	6.7
50	No retraining	120
	ADWIN	4.9
	pH	4.8
	DDM	4.3
	EDDM	4.6
	PE	4.2
	Hybrid	4.1
100	No retraining	144.5
	ADWIN	3.2
	pH	2.5
	DDM	2.6
	EDDM	38.3
	PE	2.9
	Hybrid	2.5

Table 7

Drift adaptation by ADWIN, PH, DDM, EDDM, PE, and Hybrid on various data collection frequency (Season length: 50 and 100 instances).

Season length	Algorithm	MAPE%	Number of retraining
50	ADWIN	5.6	1
	pH	5.6	1
	DDM	5.7	1
	EDDM	6.5	1
	PE	6.09	1
	Hybrid	5.83	1
100	ADWIN	3.2	1
	pH	2.5	1
	DDM	2.6	1
	EDDM	3.83	1
	PE	3.3	1
	Hybrid	2.9	1

100 instances per season. Through experiments involving various season lengths, we inspect the influence of data collection frequency, represented by season length, on the concept drift adaptation technique by retraining a model. Consequently, we aim to gain insights into how season length, so-called data collection frequency in real-world scenarios, affects the model retraining process and its subsequent implications on the prediction performance of the updated model. Table 7 displays the model's prediction error, quantified using MAPE, and the count of retraining events initiated by the concept drift detection algorithms. These outcomes are presented for two distinct scenarios: one with a season length of 50 instances and another with a season length of 100 instances. The results indicate that the initiation of a single retraining event implies the absence of false positives in detection. Furthermore, a noteworthy observation is that, under identical conditions, the dataset with 100 instances per season exhibits lower prediction error compared to the dataset with 50 instances per season. This observation underscores the significance of having a greater number of instances per season, as it enables the deep model to better capture data patterns and significantly influences model convergence.

Window size for model retraining. In this section, we extend our evaluation by exploring the drift adaptation performance in relation to the data window length employed for model updates. This analysis aims to address the question of how long it takes to attain a well-performing model for production following the detection of a drift. We delve into the complexity aspect, focusing on the time required for training the model and the waiting time for data collection. For this investigation, we utilize varying window lengths, specifically 100, 200, and up to 1000, for the model retraining process. These window lengths correspond to 1, 2, and up to 10 seasons in the dataset, respectively. Consequently, our exploration ultimately centers on exploring the trade-off between achieving minimal prediction error and rapidly obtaining a functional model for production. The performance of the model, updated using various window lengths, is visually presented in Fig. 5. One key observation is that the performance of the retrained model consistently improves as we progress from left to right across the graph, signifying an increase in the number of instances utilized. This behavior can be attributed to the fact that deep learning models require a substantial volume of data not only for initial training but also for weight updates during retraining. However, it's noteworthy that the difference in performance becomes negligible as we transition from a window size of 300 to 700. In such instances, a trade-off can be considered between the model's MAPE% and the training time, allowing for improved performance while managing computational resources more efficiently.

Window sizes between 100 and 1000 instances were selected to represent short-term and long-term retraining cycles. Smaller windows accelerate adaptation but increase noise sensitivity, whereas larger windows improve model stability at the cost of longer retraining times. Em-

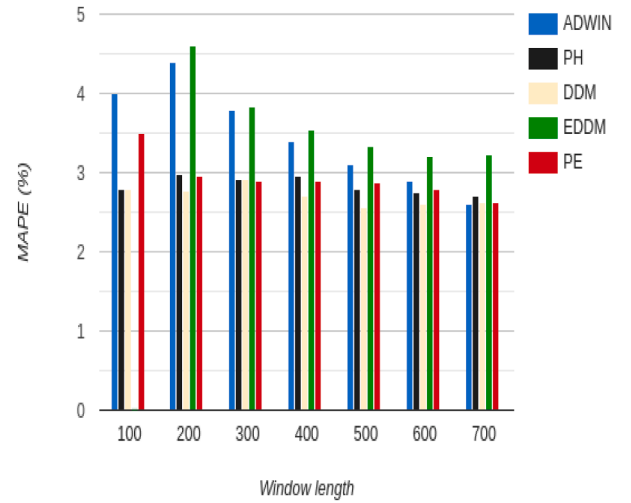


Fig. 5. Drift adaptation by ADWIN, PH, DDM, EDDM and PE and various window length utilized for model retraining.

pirically, performance stabilized beyond a window size of 700, suggesting an optimal trade-off between accuracy and computational efficiency.

5.2. Real-world workload 1

5.2.1. Concept drift detection

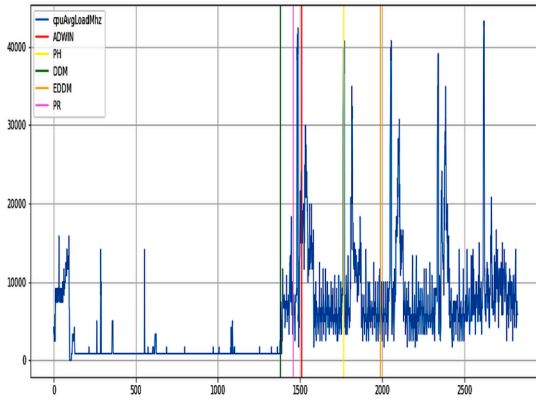
To evaluate the effectiveness of concept drift detection methods using real-world workload data, we first apply these algorithms to the entire workload traces collected from multiple servers. Next, we assess the performance of each algorithm by analyzing its capacity to promptly detect new concept shifts while minimizing false positives. Fig. 6 offers a visual representation of the specific instances where each algorithm detected concept drift in both datasets. As demonstrated in Fig. 6(a), in the first dataset, PE, DDM, and Adwin exhibited a tendency to detect concept drift earlier than pH and EDDM. However, EDDM displayed a considerable delay in detecting concept drift. This delay might be attributed to the fact that DDM, PE, and EDDM rely on monitoring model errors, with EDDM specifically tracking two distinct errors, which can introduce delays in the detection process.

Furthermore, Table 8 provides insights into the number of retraining instances triggered by each concept drift detection method and the corresponding count of false positives. Notably, all methods, except for DDM, effectively identified changes in the workload without producing any false positives. The limitation observed with DDM could potentially be attributed to the necessity for fine-tuning its threshold parameters to align with the specific characteristics of the dataset being utilized. In conclusion, our application of data mining algorithms, including ADWIN, DDM, PH, EDDM, PE, and the hybrid method, proved effective in detecting shifts in real-world server workloads, ensuring their suitability for identifying changes in cloud workloads.

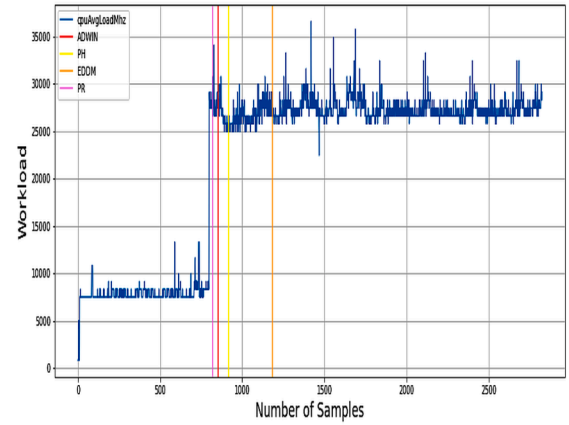
5.2.2. Concept drift adaptation

In order to assess the adaptability of the implemented method to concept drift using real-world datasets, we introduced model retraining with varying window sizes and diverse data types. The retraining process is primarily contingent on decisions made by concept drift detection algorithms. Consequently, we integrate each algorithm with the base learner to establish an adaptive approach for workload prediction. This evaluation encompasses two distinct components.

Initially, we partitioned the dataset into two segments: *before detection* and *after detection*. Following the identification of a concept drift, we experimented with varying ratios of the *after detection* data to *before detection* data, spanning from 10% to 100%, to update the existing model. This evaluation serves several purposes, such as assessing the value of



(a) Concept drift detection with CPU average workload on server 1



(b) Concept drift detection CPU average workload on server 8

Fig. 6. Detection of concept drift in real-world CPU workload data. The vertical lines denote the drift points detected by the ADWIN, PH, DDM, EDDM, and PE methods.

Table 8

Drift detection methods ADWIN, PH, DDM, EDDM, PE, and Hybrid and corresponding prediction performance and mean retraining time of model in MAPE% in contrast to the baseline.

Algorithm	Retrainings	False positives	MAPE%	Time (s)
Baseline	0	0	27.3	0
ADWIN	1	0	1.5	4.9
PH	1	0	2.3	5.1
DDM	0	0	27.3	0
EDDM	1	0	1.57	5.6
PE	1	0	1.9	5.2
Hybrid	1	0	1.76	5.0

historical data for future model updates and aiding in decisions regarding the removal of unnecessary data to conserve storage space. Fig. 7(a) provides a visualization of the percentage of *after detection* data used for model retraining and the corresponding Mean Absolute Percentage Error (MAPE) achieved. Notably, the performance of the model consistently improves as the percentage of “after detection” data used for retraining increases from 10% to 100%. This trend underscores that the algorithms do not exhibit significant delays in identifying new concepts with the *before detection* data, thus facilitating effective model updates.

The second evaluation includes an assessment of the window size used for the model update. As illustrated in Fig. 7(b), the model updated with window sizes 100, 200, and 300 showed fluctuations in MAPE% in all algorithms. However, prediction error became steady as the window size set to 500 and demonstrates very small change as window size increases to 1000. From both the above assessments, we can conclude that the adaptive learner need to be set with suitable parameters on window size and % of *after detection* data. To enforce this assessment, Table 8 illustrates the retraining performance achieved by concept drift detection methods in comparison to base learner and retraining time. It can be observed that the retraining process improved the model prediction performance by more than 80% in all cases. Parameter setting for window size depends on the end-goal of the prediction model. In situations where accurate predictions are required, a larger window size can be set which can be increase waiting time until the good performing model is produced whereas on times when only direction of the workload increase or decrease need to be known smaller window sizes can be selected which in turn can be achieved in shorter time.

To assess the efficiency of the two adaptation techniques employed, Table 9 demonstrates that at each stage of drift detection in the work-

load, both adaptation methods are applied: retraining the existing model with new data by updating the previously trained weights and training a completely new model from scratch. Consequently, Table 9 presents the MAPE (%) for both methods, along with the specific change detection methods used, in comparison to the baseline approach of static workload prediction. In every scenario, the change adaptation methods consistently enhanced the performance of the static baseline model.

5.3. Real-world workload 2

5.3.1. Concept drift detection

To assess the effectiveness of concept drift detection algorithms in large-scale real-world workload traces, we performed a series of experiments with five drift detection algorithms on CPU and memory utilization metrics. In real-world datasets, the concept drift occurrence position is not known. Thus, we compare the outputs of the concept drift algorithms in addition to the model performance to evaluate them. Table 12 highlights the number of concept drift detections by algorithms ADWIN, PH, DDM, EDDM, PE, and Hybrid on memory utilization. The results obtained show that all algorithms detected drift in the data, apart from EDDM. The number of drifts obtained corresponds to the number of retraining to the initial model. The relative number of detections by PH, DDM, and PE are similar. However, ADWIN detected additional drifts in the data. Hybrid method has surpassed by initiating one retraining with no false positives and obtaining the best model performance.

5.3.2. Concept drift adaptation

We selected CPU and memory utilization metrics as the basis for training our models. The baseline model was trained using 10,080 records, equivalent to one week of data. To assess how well the models adapt to changes in workload through retraining, driven by concept drift detection decisions made by the selected algorithms, we experimented with various window sizes. These sizes included 180, 360, 720, 1440, and 2880 instances, corresponding to data records spanning from three hours to two days, with data saved at one-minute intervals.

As illustrated in Fig. 8, increasing the window size led to a slight improvement in the retrained model’s performance across all cases when utilizing the concept drift detection algorithms ADWIN, PH, DDM, and PE. However, there were two instances where the error slightly increased when using 1440 points. These correspond to the positions where the algorithms detected drift.

To further substantiate this assessment, Tables 10 and 11 present the retraining performance of the concept drift detection algorithms com-

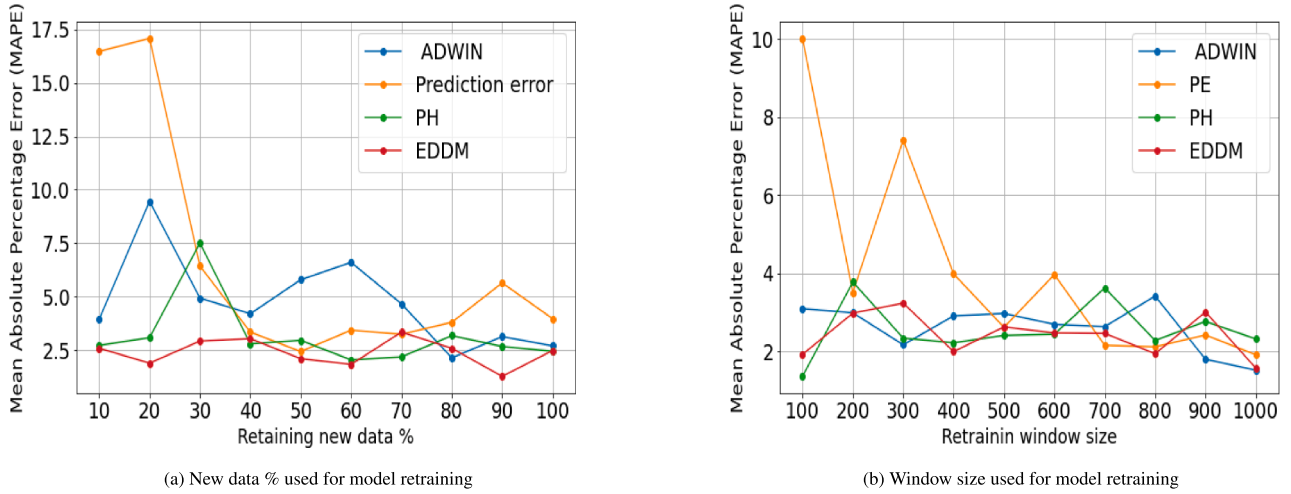


Fig. 7. Concept drift adaptation: retraining data type utilized and window size utilized for retraining and MAPE% on real-world workload dataset, change detection by methods: ADWIN, PH, DDM, EDDM, and PE.

Table 9

Drift detection methods ADWIN, PH, DDM, EDDM, PE, and Hybrid and corresponding prediction performance of retrained model in MAPE% in contrast with new model training at point of change detection real-world workload dataset 1.

Retraining Workload	Retraining MAPE (%)							New training Workload	New training MAPE (%)						
	Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid		Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid
Server 2	11.9	3.1	2.9	4.2	4.8	2.35	2.30	Server 2	11.9	3.5	3.1	3.9	4.1	2.89	2.79
Server 8	9.45	2.6	2.7	2.63	3.8	2.63	2.52	Server 8	9.45	2.8	2.1	2.4	3.9	2.1	2.13

Table 10

Drift detection methods ADWIN, PH, DDM, EDDM, PE, and Hybrid along with corresponding prediction performance of retrained model in MAPE% in contrast with new model training at point of change detection for memory utilization real-world workload dataset 2.

Workload	Retraining NRMSE							New training Workload	New training NRMSE						
	Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid		Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid
Memory utilization 1	0.31	0.09	0.08	0.1	0.31	0.08	0.01	Memory utilization 1	0.31	0.09	0.1	0.1	0.31	0.08	0.01
Memory utilization 2	2.6	0.8	0.4	0.4	0.1	0.2	0.21	Memory utilization 2	2.6	0.9	0.9	1.5	0.1	0.09	0.08

Table 11

Drift detection methods ADWIN, PH, DDM, EDDM, PE, and Hybrid and corresponding prediction performance of retrained model in NRMSE in contrast with new model training at point of change detection for CPU utilization in real-world workload dataset 2.

Workload	Retraining NRMSE							New training Workload	New training NRMSE						
	Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid		Baseline	ADWIN	PH	DDM	EDDM	PE	Hybrid
CPU utilization 1	0.180	0.060	0.045	0.050	0.050	0.069	0.062	CPU utilization 1	0.18	0.056	0.036	0.039	0.032	0.061	0.059
CPU utilization 2	0.237	0.051	0.010	0.10	0.237	0.027	0.010	CPU utilization 2	0.237	0.039	0.030	0.023	0.237	0.021	0.027

pared to the base learner trained with historical data, focusing on CPU and memory utilization. We employed the NRMSE metric to compare model performance in CPU utilization metrics, due to the fact that MAPE provides indefinite results for values close to zero.

It is evident from the tables that the model retraining approach employed in our experiments consistently improved model performance across all cases. For scenarios requiring timely predictions, smaller window sizes can be selected to achieve updated models with minimal delays. Conversely, when precision in predictions is paramount, larger window sizes can be utilized, even though this may entail waiting for an extended duration until sufficient data is collected to facilitate the model retraining process.

Finally, to study the impact of various model update options on changes in workload, we performed a series of experiments in which the model is updated by retraining to update the parameters and by training a new model from scratch. This step assists in investigating the two adaptation methods utilized based on model performance and time overhead. MAPE (%) of updated models are illustrated in Table 10, where both

Table 12

Drift detection methods ADWIN, PH, DDM, EDDM, PE, and Hybrid methods and corresponding prediction performance and training time of retrained model in MAPE% in contrast to the baseline model.

Algorithm	Number of retraining	MAPE%	Mean Retraining time (s)
Baseline	0	0.31	0
ADWIN	7	0.09	5.3
PH	2	0.08	5.2
DDM	3	0.1	5.8
EDDM	0	0.31	0
PE	3	0.08	5.42
Hybrid	1	0.05	5.1

adaptation strategies updated the baseline model performance. However, it must be noted that generating a newly needed prediction model on drift detection requires a vast amount of training data, which in turn results in a longer waiting time for data to arrive in the system. To sum-

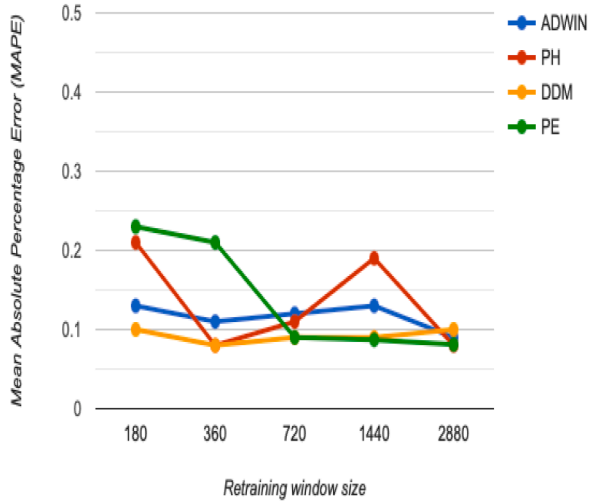


Fig. 8. Concept drift adaptation: data type and window size utilized for retraining and MAPE% on real-world workload data, change detection by methods: ADWIN, PH, DDM, EDDM, and PE.

marize, we found that, utilizing concept drift algorithms to assist the model update process utilized in workload prediction guarantees an improvement in performance in all cases. We also learned, selecting from the list of parameters including window size, data type for retraining and model update strategy depends on factors, which confirms there is a tradeoff between the best performance and delay in achieving the updates. This is because the time required to achieve the updated model is directly proportional to the time it takes for the system to generate data, multiplied by the time needed to train the model.

We Added statistical significance analysis

5.4. Statistical significance and variability analysis

To assess the reliability and reproducibility of the reported results, each experiment was executed five times using different random seeds. This number of repetitions was selected as a practical compromise between statistical rigor and computational cost, since preliminary tests showed negligible variance beyond five runs. For each configuration, we report the mean performance along with the standard deviation (SD) and 95% confidence intervals (CI). The confidence intervals were computed using the Student's t -distribution as follows:

$$CI = \bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (1)$$

where $\bar{x}$ denotes the sample mean, s is the standard deviation across runs, and n is the number of repetitions.

Pairwise statistical significance between the baseline and adaptive models was tested using a paired t -test. Across all datasets, the proposed hybrid method achieved statistically significant improvements ($p < 0.01$). For instance, on the real-world dataset 1, the hybrid method yielded a mean improvement of $60.8\% \pm 4.5\%$ (95% CI) over the baseline, confirming that the observed enhancement is both robust and reproducible.

Reviewer 2, comment 2: We fully agree and thank the reviewer for highlighting this omission. We have now added a dedicated ablation study in Section 5.5. The study evaluates three variants: data-only detection (statistical monitoring), model-error-only detection, and the proposed hybrid method.

The new Table 13 reports prediction accuracy (MAPE) and false positive rates across multiple runs. The results clearly demonstrate that the hybrid method achieves lower false positives and superior accu-

Table 13

Compact ablation study results (mean $\pm$ SD over five runs). The hybrid detector achieves higher accuracy and lower false positive rate (FPR) than single-component variants.

Method	MAPE (%)	$\pm$ SD	FPR (%)	Time (s)
Data-only	5.7	0.4	8.3	4.9
Error-only	5.3	0.5	12.6	4.8
Hybrid	4.1	0.3	2.1	5.0

racy, quantitatively validating the benefit of combining data-based and performance-based signals.

5.5. Ablation study

To evaluate the contribution of each component in the proposed hybrid concept drift detection framework, we conducted an ablation study comparing three variants:

- **Data-only detection:** uses statistical monitoring (ADWIN) without model performance feedback.
- **Model error-only detection:** triggers retraining solely based on prediction error deviation.
- **Hybrid detection (proposed):** combines both statistical and model error-based indicators.

Each configuration was evaluated using both synthetic and real-world cloud workload datasets under identical conditions. All experiments were repeated five times with different random seeds, and we report the mean values along with the standard deviation (SD) and 95% confidence intervals (CIs). Table 13 summarizes the average Mean Absolute Percentage Error (MAPE), False Positive Rate (FPR), and retraining time for each configuration.

The results demonstrate that the hybrid detector consistently outperforms both single-component variants. Compared with the data-only detector, the hybrid method improved prediction accuracy by $8.3\% \pm 2.1\%$ (95% CI) and reduced false positives by approximately 65%. The paired t -test confirmed that these improvements were statistically significant ($p < 0.01$).

Moreover, the hybrid approach maintained comparable retraining costs (around 5 s per update), confirming its efficiency for real-time adaptation. These findings validate the design hypothesis that integrating complementary drift indicators-data-level and model-level-provides more reliable and robust drift detection in dynamic cloud environments.

6. Conclusion

In this work, we present efficient retraining strategies for machine learning models used in cloud management systems, focusing on adaptive cloud workload prediction through concept drift detection. The high variability of workloads in cloud environments causes machine learning models trained on historical data to become outdated over time. While most existing research emphasizes accuracy in predictions, it often overlooks the need for adaptive models. Workload datasets frequently experience statistical shifts in data distribution, known as concept drift, which can degrade model performance. Detecting these shifts and updating models accordingly is essential. Although several methods in the literature can detect changes in streaming data, they often suffer from delays and false positive detections, leading to inefficient retraining and increased computational costs.

To address this issue, we propose a hybrid concept drift detection approach that combines stream data processing with model performance monitoring. This approach addresses the issues of delayed adaptation and reduces false positive detections. The experiments, conducted on

synthetic and real-world workload datasets, indicate that initiating retraining based on drift detectors consistently enhances overall performance. On average, these methods yield approximately a 90% improvement in model prediction performance for synthetic datasets and a 60% improvement for real-world datasets when compared to the baseline static model that did not undergo retraining.

Furthermore, the choice of data for retraining, including considerations of data granularity, window length, and data type, undeniably influences the retraining process. In our experiments, we categorize data as *before detection* and *after detection*. The most favorable outcomes are observed when a substantial portion of the dataset used for retraining consists of *after detection* data. This preference stems from the fact that data collected after drift detection contains substantial information about the new concept, enabling the updating of the existing model. It's important to highlight this requires waiting for an adequate amount of post-detection data to accumulate. The proposed hybrid retraining method partially solves this issue by beginning to accumulate data upon warning rather than waiting to accumulate after drift detection.

While this study focuses on cloud workload prediction, the proposed hybrid drift detection framework is generalizable to other dynamic environments such as Internet of Things (IoT) systems and edge computing platforms. These domains share similar data drift characteristics, suggesting that the proposed approach could enhance adaptive modeling and predictive reliability in broader real-time systems. Our future work includes investigating ensemble-based adaptation strategies that combine multiple predictive models to enhance robustness against diverse drift patterns. Ensemble retraining is expected to further improve resilience to noise and heterogeneous workloads.

CRedit authorship contribution statement

Lidia Kidane: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Paul Townend:** Writing – review & editing, Supervision, Conceptualization; **Thijs Metsch:** Writing – review & editing, Conceptualization; **Erik Elmroth:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

Financial support has been provided in part by the Knut and Alice Wallenberg Foundation under grant KAW 2019.0352 and by the eSSSENCE Programme under the Swedish Government's Strategic Research Initiative. This work uses data provided by the WARA-Ops (waraps.org) initiative.

References

- [1] T. Dillon, C. Wu, E. Chang, Cloud computing: issues and challenges, in: 2010 24th IEEE International Conference on Advanced Information Networking and Applications, 2010, pp. 27–33. <https://doi.org/10.1109/AINA.2010.187>
- [2] N. Herbst, S. Kounev, R. Reussner, Elasticity in cloud computing: what it is, and what it is not, *Int. Conf. Auton. Comput.* (2013) 23–27.
- [3] Y. Jiang, C.-S. Perng, T. Li, R.N. Chang, Cloud analytics for capacity planning and instant VM provisioning, *IEEE Trans. Netw. Serv. Manage.* 10 (3) (2013) 312–325. <https://doi.org/10.1109/TNSM.2013.051913.120278>
- [4] A. Saboor, M.F. Hassan, R. Akbar, S.N.M. Shah, F. Hassan, S.A. Magsi, M.A. Siddiqui, Containerized microservices orchestration and provisioning in cloud computing: a conceptual framework and future perspectives, *Appl. Sci.* 12 (12) (2022). <https://doi.org/10.3390/app12125793>
- [5] M. Fragkoulis, P. Carbone, V. Kalavri, A. Katsifodimos, A Survey on the Evolution of Stream Processing Systems, *CoRR abs/2008.00842* (2020). [arXiv:2008.00842](https://arxiv.org/abs/2008.00842)
- [6] D. Saxena, J. Kumar, A.K. Singh, S. Schmid, Performance analysis of machine learning centered workload prediction models for cloud, *IEEE Trans. Parallel Distrib. Syst.* 34 (4) (2023) 1313–1330. <https://doi.org/10.1109/TPDS.2023.3240567>
- [7] M. Maisenbacher, M. Weidlich, Handling concept drift in predictive process monitoring, in: 2017 IEEE International Conference on Services Computing (SCC), 2017, pp. 1–8. <https://doi.org/10.1109/SCC.2017.10>
- [8] R.P. J.C. Bose, W.M.P. van der Aalst, Ž. Indrė, M. Pechenizkiy, Handling concept drift in process mining, in: H. Mouratidis, C. Rolland (Eds.), *Advanced Information Systems Engineering*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 391–405.
- [9] G. Widmer, M. Kubat, Learning in the presence of concept drift and hidden contexts, *Mach. Learn.* 23 (1994). <https://doi.org/10.1007/BF00116900>
- [10] J. Gama, et al., A survey on concept drift adaptation, *ACM Comp. Surv.* 46 (4) (2014). <https://doi.org/10.1145/2523813>
- [11] F. Bayram, B.S. Ahmed, A. Kessler, From concept drift to model degradation: an overview on performance-aware drift detectors, *Knowl. Based Syst.* 245 (2022) 108632. <https://doi.org/10.1016/j.knosys.2022.108632>
- [12] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: a review, *IEEE Trans. Knowl. Data Eng. PP* (2018) 1. <https://doi.org/10.1109/TKDE.2018.2876857>
- [13] Q. Xiang, L. Zi, X. Cong, Y. Wang, Concept drift adaptation methods under the deep learning framework: a literature review, *Appl. Sci.* 13 (11) (2023). <https://doi.org/10.3390/app13116515>
- [14] H. Toumi, Z. Brahmi, M.M. Gammoudi, RTSLPS: real time server load prediction system for the ever-changing cloud computing environment, *J. King Saud Univ. Comput. Inf. Sci.* 34 (2) (2022) 342–353. <https://doi.org/10.1016/j.jksuci.2019.12.004>
- [15] B. Shayesteh, C. Fu, A. Ebrahimzadeh, R. Glitho, Auto-adaptive fault prediction system for edge cloud environments in the presence of concept drift, in: 2021 IEEE International Conference on Cloud Engineering (IC2E), 2021, pp. 217–223. <https://doi.org/10.1109/IC2E52221.2021.00037>
- [16] S. Aranovskiy, A. Bobtsov, A. Kremlev, N. Nikolaev, O. Slita, Identification of frequency of biased harmonic signal, *Eur. J. Control* 16 (2007). <https://doi.org/10.3166/ejc.16.129-139>
- [17] E. Elmroth, J. Tordsson, F. Hernández, A. Ali-Eldin, P. Svård, M. Sedaghat, W. Li, Self-management challenges for multi-cloud architectures, in: W. Abramowicz, I.M. Llorente, M. Surridge, A. Zisman, J. Vayssière (Eds.), *Towards a Service-Based Internet*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 38–49.
- [18] L. Kidane, P. Townend, T. Metsch, E. Elmroth, When and how to retrain machine learning-based cloud management systems, in: 2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2022, pp. 688–698. <https://doi.org/10.1109/IPDPSW55747.2022.00120>
- [19] J. Cao, J. Fu, M. Li, J. Chen, CPU load prediction for cloud environment based on a dynamic ensemble model, *Softw. Pract. Exper.* 44 (7) (2014) 793–804. <https://doi.org/10.1002/spe.2231>
- [20] C. Liu, C. Liu, Y. Shang, S. Chen, B. Cheng, J. Chen, An adaptive prediction approach based on workload pattern discrimination in the cloud, *J. Netw. Comput. Appl.* 80 (C) (2017) 35–44. <https://doi.org/10.1016/j.jnca.2016.12.017>
- [21] Y. Yu, V. Jindal, F. Bastani, F. Li, L.-L. Yen, Improving the smartness of cloud management via machine learning based workload prediction, in: 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 02, 2018, pp. 38–44. <https://doi.org/10.1109/COMPSAC.2018.10200>
- [22] Q. Yang, Y. Zhou, Y. Yu, J. Yuan, X. Xing, S. Du, Multi-step-ahead host load prediction using autoencoder and echo state networks in cloud computing, *J. Supercomput.* 71 (8) (2015) 3037–3053. <https://doi.org/10.1007/s11227-015-1426-8>
- [23] B. Song, Y. Yu, Y. Zhou, Z. Wang, S. Du, Host load prediction with long short-term memory in cloud computing, *J. Supercomput.* 74 (12) (2018) 6554–6568. <https://doi.org/10.1007/s11227-017-2044-4>
- [24] S. Kumar, N. Muthiyar, S. Gupta, D.A. Dinesh, A. Nigam, Association learning based hybrid model for cloud workload prediction, in: 2018 International Joint Conference on Neural Networks (IJCNN) (2018) 1–8.
- [25] J. Gao, H. Wang, H. Shen, Task failure prediction in cloud data centers using deep learning, *IEEE Trans. Serv. Comput. PP* (2020) 1. <https://doi.org/10.1109/TSC.2020.2993728>
- [26] K. Mason, M. Duggan, E. Barrett, J. Duggan, E. Howley, Predicting host CPU utilization in the cloud using evolutionary neural networks, *Future Gener. Comput. Syst.* 86 (2018). <https://doi.org/10.1016/j.future.2018.03.040>
- [27] M. Duggan, R. Shaw, J. Duggan, E. Howley, E. Barrett, A multistep-ahead prediction approach for scheduling live migration in cloud data centers, *Softw. Pract. Exper.* 49 (2018). <https://doi.org/10.1002/spe.2635>
- [28] Z. Chen, J. Hu, G. Min, A.Y. Zomaya, T. El-Ghazawi, Towards accurate prediction for high-dimensional and highly-variable cloud workloads with deep learning, *IEEE Trans. Parallel Distrib. Syst.* 31 (4) (2020) 923–934. <https://doi.org/10.1109/TPDS.2019.2953745>
- [29] D. Saxena, A.K. Singh, Auto-adaptive learning-based workload forecasting in dynamic cloud environment, *Int. J. Comput. Appl.* 44 (6) (2022) 541–551. <https://doi.org/10.1080/1206212X.2020.1830245>
- [30] L. Bao, J. Yang, Z. Zhang, W. Liu, J. Chen, C. Wu, On accurate prediction of cloud workloads with adaptive pattern mining, *J. Supercomput.* (2022). <https://doi.org/10.1007/s11227-022-04647-5>

- [31] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: a review, *IEEE Trans. Knowl. Data Eng.* (2018) 1. <https://doi.org/10.1109/tkde.2018.2876857>
- [32] T.G. Dietterich, et al., Ensemble learning, *Handb. Brain Theory Neural Netw.* 2 (1) (2002) 110–125.
- [33] H. McKay, Online transfer learning for concept drifting data streams, Ph.D. thesis, University of Warwick, 2022.
- [34] C. Yang, Y.-M. Cheung, J. Ding, K.C. Tan, Concept drift-tolerant transfer learning in dynamic environments, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (8) (2022) 3857–3871. <https://doi.org/10.1109/TNNLS.2021.3054665>
- [35] L. Baier, J. Reimold, N. Kühl, Handling concept drift for predictions in business process mining, in: 2020 IEEE 22nd Conference on Business Informatics (CBI), 1, IEEE, 2020, pp. 76–83.
- [36] M. Lima, T.S. Filho, R.A. de A. Fagundes, A comparative study on concept drift detectors for regression, in: A. Britto, K. Valdivia Delgado (Eds.), *Intelligent Systems*, Springer International Publishing, Cham, 2021, pp. 390–405.
- [37] H. Touni, Z. Brahmi, M.M. Gammoudi, Extended hoeffding adaptive tree based-server load prediction in cloud computing environment, in: Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region, HP-Asia '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 161–168. <https://doi.org/10.1145/3368474.3368475>
- [38] A. Mallick, K. Hsieh, B. Arzani, G. Joshi, Matchmaker: data drift mitigation in machine learning for large-scale systems, *Proc. Mach. Learn. Syst.* 4 (2022) 77–94.
- [39] M. Amiri, L. Mohammad-Khanli, R. Mirandola, An online learning model based on episode mining for workload prediction in cloud, *Future Gener. Comput. Syst.* 87 (2018) 83–101. <https://doi.org/10.1016/j.future.2018.04.044>
- [40] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (1997) 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [41] A. Graves, Supervised sequence labelling with recurrent neural networks, in: *Studies in Computational Intelligence*, 2012. <https://api.semanticscholar.org/CorpusID:2118350>
- [42] Z. Han, J. Zhao, H. Leung, K.F. Ma, W. Wang, A review of deep learning models for time series prediction, *IEEE Sens. J.* 21 (6) (2021) 7833–7848. <https://doi.org/10.1109/JSEN.2019.2923982>
- [43] T. Shi, K. Shide, A comparative analysis of LSTM, GRU, and transformer models for construction cost prediction with multidimensional feature integration, *J. Asian Archit. Build. Eng.* (2025) 1–16.
- [44] S. Siami-Namini, N. Tavakoli, A.S. Namin, A comparison of ARIMA and LSTM in forecasting time series, in: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), IEEE, 2018, pp. 1394–1401.
- [45] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: *SDM*, 2007. <https://api.semanticscholar.org/CorpusID:2279539>
- [46] M. Baena-García, J. Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavaldà, R. Morales-Buono, Early drift detection method (2006).
- [47] R.C. Cavalcante, A.L.L. Oliveira, An approach to handle concept drift in financial time series based on extreme learning machines and explicit drift detection, in: 2015 International Joint Conference on Neural Networks (IJCNN), 2015, pp. 1–8. <https://doi.org/10.1109/IJCNN.2015.7280721>
- [48] S. Schelter, F. Biessmann, T. Januschowski, D. Salinas, S. Seufert, G. Szarvas, On challenges in machine learning model management, *IEEE Data Eng. Bull.* (2015). <https://www.amazon.science/publications/on-challenges-in-machine-learning-model-management>
- [49] J.P. Barddal, H.M. Gomes, F. Enembreck, B. Pfahringer, A survey on feature drift adaptation: definition, benchmark, challenges and future directions, *J. Syst. Softw.* 127 (2016). <https://doi.org/10.1016/j.jss.2016.07.005>
- [50] J. Montiel, J. Read, A. Bifet, T. Abdesslem, Scikit-multiflow: a multi-output streaming framework, *J. Mach. Learn. Res.* 19 (1) (2018) 2915.
- [51] L. Kidane, P. Townend, T. Metsch, E. Elmroth, Automated hyperparameter tuning for adaptive cloud workload prediction, in: Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing, 2023, pp. 1–8.



Lidia Kidane is a PhD candidate in Computing Science at Umeå University, Sweden. She obtained an MSc in Machine Intelligence from the African Institute for Mathematical Science (AIMS), Rwanda in 2020 and a Bachelor of Science in Computer Engineering from Eritrea Institute of Technology in 2014. Her research interests include Machine learning, Big data analytics, and Cloud computing.



Paul Townend is an Associate Professor at Umeå University, Sweden. He is the founder of the Green Distributed Computing group at Umeå, and has led/co-led over \$5M in successfully completed research projects. Paul is currently Scientific Coordinator of a major Horizon Europe project (COGNIT) and Co-Manager of a national data infrastructure in Sweden (WARA-Ops). He is also a member of the WASP Graduate School Management group in Sweden, overseeing more than 460 PhD students nationwide.



Thijs is a Research Engineer at Intel Labs, specializing in system performance, distributed systems. With a background in HPC, Grids, and Cloud/Edge technologies, he has worked for industry leaders like IBM, Sun Microsystems, and the German Aerospace Center. Throughout his career, Thijs has simplified shipbuilding, executed massive parallel workloads, and optimized large-scale hybrid compute environments for both performance and efficiency. He also contributed to the creation of one of the first cloud computing standards over a decade ago. Currently, he focuses on enhancing orchestration tools like Kubernetes, integrating AI/ML techniques to drive efficiency and sustainability.



Erik Elmroth is Professor and has been department head and deputy head for thirteen years and Deputy Director for a national supercomputer center for another thirteen years. He has established the Umeå University research on distributed systems. Elmroth is part of management for the 550 M Euro Wallenberg AI, Autonomous Systems and Software Program (WASP) and member of the program management group for the eSENCE Strategic Research Program. Previously, he was principal investigator for Cloud Control, which was the second largest project ever funded by the [Swedish Research Council](#). Elmroth is member of the Swedish Royal Academy for Engineering Sciences. Previously, he was the Chair of the Board of the Swedish National Infrastructure for Computing; Chair of the Swedish Research Council's (VR's) expert group on e-science infrastructures, and member of VR's Council for Research Infrastructures (RFI). He has been appointed by the Nordic Council of Ministers for developing two international research strategies. He received the Nordea Scientific Award 2011. Pre-historic highlights include being co-winner of the SIAM Linear Algebra Prize 2000, for the most outstanding linear algebra publication world-wide (in any journal) during the preceding 3-year period. International experiences include a year at NERSC, Lawrence Berkeley National Laboratory and one semester at the MIT. He is founder and chair of the Control Workshops series. Elmroth has co-founded Elastisys, an expert on security and regulatory compliance in the cloud-native ecosystem.